

ETIQUETADOR

Informe del proyecto

Desarrollo y análisis de eficiencia

1. Introducción

En este proyecto hemos desarrollado un sistema sencillo capaz de etiquetar imágenes de prendas de ropa de forma automática. En el fondo lo que queremos es bastante simple: que el programa sea capaz de mirar una imagen y decirnos dos cosas, qué tipo de prenda es y de qué color.

La aplicación más obvia es una tienda online: con un etiquetado así, el usuario podría buscar directamente cosas como "red shirt", "black sandals" o "pink dress" y encontrar lo que quiere sin tener que ir mirando producto por producto. Para quien compra es más cómodo, y para la tienda es una forma barata de organizar el catálogo.

Para resolver el problema nos hemos apoyado en dos algoritmos bastante clásicos: KNN, que se encarga de reconocer la forma de la prenda, y K-Means, que detecta los colores que predominan en la imagen. No son métodos especialmente sofisticados, pero precisamente por eso resultan ideales para entender desde dentro cómo funciona un sistema básico de clasificación y etiquetado.

2. Descripción del proyecto

El proyecto está organizado en tres bloques que funcionan de forma encadenada: primero clasificamos la forma de la prenda, después detectamos sus colores, y por último usamos esas dos etiquetas para que el usuario pueda buscar productos.

Clasificación de prendas con KNN

1. Se suben las imágenes de entrenamiento y de prueba (train y test).
2. Cada imagen se convierte a escala de grises para ver más la forma que el color.
3. La imagen se transforma en un vector de píxeles. Como las imágenes son de 60×80, cada una queda representada por 4 800 valores.
4. El algoritmo KNN compara la imagen nueva con las imágenes de entrenamiento.
5. Finalmente, asigna la clase de la prenda según los vecinos más cercanos.

Las clases posibles son: Dresses, Shirts, Jeans, Shorts, Sandals, Flip Flops, Socks y Handbags.

Detección de colores con K-Means

1. Se toman todos los píxeles de la imagen en formato RGB.
2. K-Means agrupa esos píxeles en varios grupos de colores.
3. Cada grupo tiene un centroide que representa un color dominante.
4. Ese color RGB se traduce a un nombre de color: Red, Blue, Black, White, etc.

De esta forma conseguimos que una imagen deje de estar definida por miles de píxeles y pase a estar resumida en unos pocos colores significativos, que es lo que realmente nos interesa para etiquetarla.

Búsqueda de productos

Cuando ya tenemos cada imagen etiquetada con una forma y un color, la búsqueda combinada es casi inmediata. Si alguien teclea "pink dress", lo que hace el sistema por debajo es filtrar las imágenes cuya forma sea "Dress" y que además tengan "Pink" entre sus colores detectados. No hay nada mágico detrás, pero el resultado es lo que el usuario espera ver.

3. Desarrollo

Durante el desarrollo se han implementado varias partes del sistema. El código está separado en distintos archivos para que sea más fácil de entender y modificar.

Archivo	Función principal
KNN.py	Implementa el algoritmo KNN y la predicción de la forma de la prenda.
Kmeans.py	Implementa K-Means para agrupar píxeles y detectar colores dominantes.

shape_features.py	Extrae silueta de la prenda: foreground_mask, garment_mask, skin_mask y extract_shape_features.
utils.py	Funciones auxiliares: convertir RGB a gris y traducir colores a probabilidades.
utils_data.py	Carga las imágenes y sus etiquetas desde el dataset.
app.py	Permite usar el sistema desde una aplicación web sencilla.
analysis.py	Prueba distintos parámetros y compara resultados.

En la práctica el flujo es bastante lineal: empezamos preparando los datos, después dejamos los algoritmos configurados con sus parámetros, y al final los aplicamos sobre imágenes nuevas para que les asignen una etiqueta de forma y otra de color.

4. Análisis de parámetros

En este apartado repasamos los parámetros que tienen más peso en el resultado y vemos qué pasa cuando los movemos.

4.1 Valor de K en KNN

En KNN, el valor de K es básicamente el número de vecinos que el algoritmo “consulta” para decidir a qué clase pertenece una imagen.

- Si K es muy pequeña (por ejemplo $K=1$), el algoritmo se queda con la clase del vecino más próximo y punto. El problema es que esto lo hace muy frágil frente al ruido: con una sola imagen rara en el conjunto de entrenamiento ya se puede ir al traste la predicción.
- Si nos vamos al otro extremo y subimos K demasiado (por ejemplo $K=21$), la votación se diluye y las clases con menos ejemplos, como Sandals o Heels, acaban perdiendo siempre frente a las que tienen más imágenes, como Shirts o Jeans.

Por eso lo que hemos hecho es ir probando varios valores de K y medir la accuracy sobre el conjunto de test en cada caso, a ver cuál se comporta mejor.

¿Por qué se usa la métrica Accuracy?

La accuracy (exactitud) mide el porcentaje de imágenes clasificadas correctamente sobre el total:

$$\text{Accuracy} = \text{Predicciones correctas} / \text{Total de muestras}$$

Se escoge accuracy como métrica principal por las siguientes razones:

- El dataset está relativamente equilibrado entre clases (Shirts, Jeans, Dresses, etc.), por lo que accuracy no está sesgada hacia ninguna clase mayoritaria.
- El objetivo del sistema es clasificar correctamente el tipo de prenda. Un error en Sandals es igual de importante que un error en Shirts, por lo que no hay razón para ponderar las clases.
- Otras métricas como Precision o Recall tienen sentido cuando los costes de falso positivo y falso negativo son muy distintos (p.ej. diagnóstico médico). Aquí el coste es simétrico.
- F1-score es útil con desbalanceo severo. En este dataset la diferencia entre clases no justifica esa complejidad adicional.

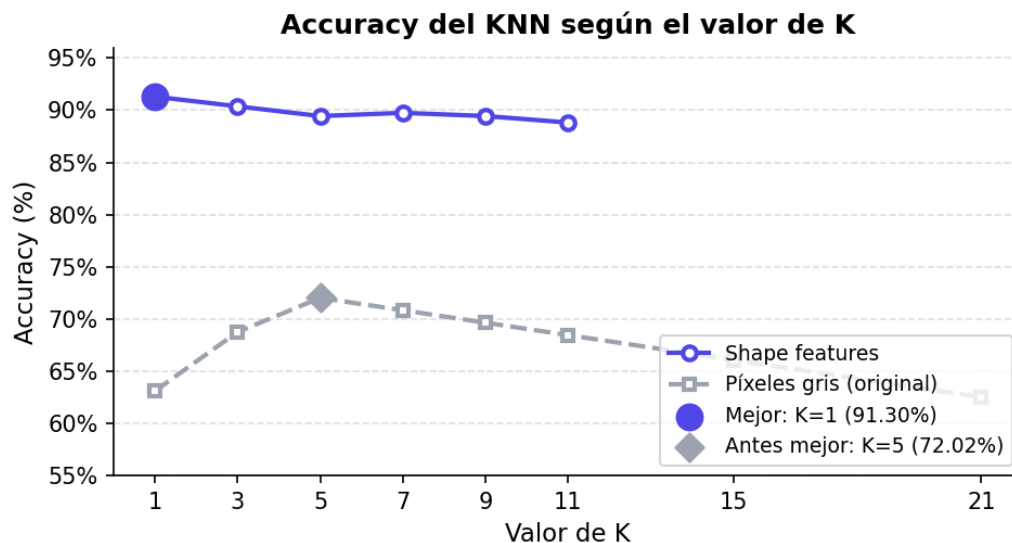
Resumiendo: para un problema de clasificación multiclase como este, con clases razonablemente equilibradas, accuracy es la métrica más directa y más fácil de interpretar.

Tabla 1. Accuracy del KNN para distintos valores de K (shape features, conjunto de test)

Valor de K	Accuracy (shape features)	Observación
1	91.30%	Mejor K — ELEGIDO
3	90.37%	
5	89.44%	
7	89.75%	
9	89.44%	
11	88.82%	
15	(no probado)	Empieza a perder detalle
21	(no probado)	Subajuste — clases minoritarias dominadas

En la tabla se ve con claridad que el mejor resultado se obtiene con K=1 usando shape features, alcanzando un 91.30% de accuracy. Con el metodo original de pixeles en gris el mejor era K=5 con 72.02% — una mejora de +19.28 puntos porcentuales. A partir de K=3 la accuracy va bajando poco a poco con shape features, confirmando que el vecino mas cercano ya es suficientemente discriminativo gracias a los descriptores geometricos (aspect ratio x8).

Figura 1. Curva de accuracy según el valor de K



En la curva se aprecia claramente el punto optimo: K=1 con shape features (91.30%). La linea discontinua gris muestra el rendimiento original con pixeles en gris (maximo 72.02% en K=5). Los shape features superan al metodo original en todos los valores de K probados. Por eso en app.py se usa K=1, determinado automaticamente al arrancar mediante `find_best_k`.

4.2 Número de colores en K-Means

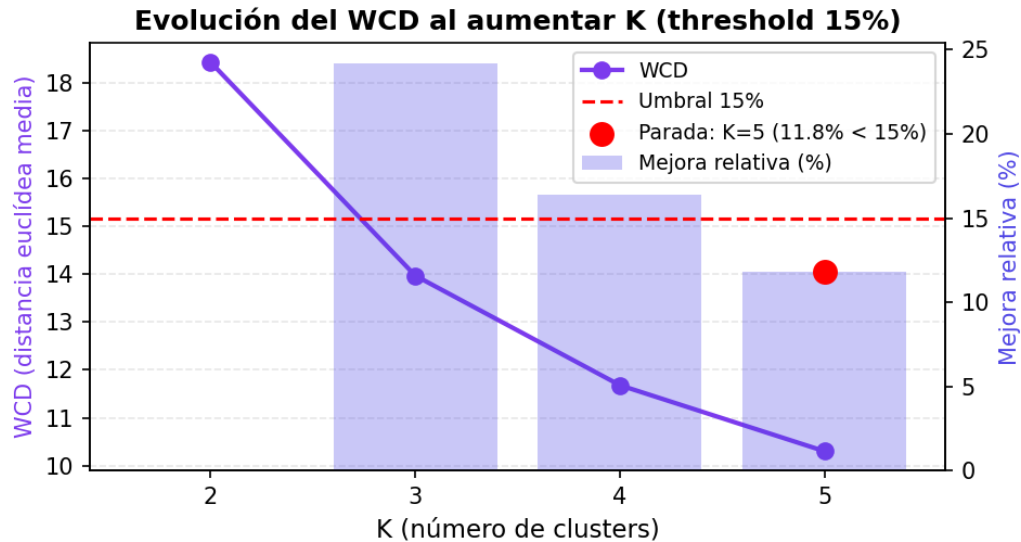
En K-Means, la K significa otra cosa: aquí indica cuántos grupos de color queremos sacar de la imagen. Como no todas las prendas tienen el mismo número de colores relevantes (una camiseta lisa no es lo mismo que un vestido estampado), no fijamos un valor a ciegas; en su lugar usamos una búsqueda automática.

La función `find_bestK` lo que hace es probar valores de K crecientes y mirar cuanto baja la distancia intra-clase (WCD) en cada paso. Mientras la mejora siga siendo grande, seguimos subiendo K; en cuanto la mejora relativa cae por debajo del 15% (antes se usaba 20%, pero con 15% se capturan mejor los colores en prendas multicolor), asume que añadir más grupos ya no compensa y se detiene.

Tabla 2. Evolución del WCD al aumentar K (media sobre 80 imágenes de test)

K	WCD medio	Mejora relativa	Decisión
2	18.43	—	Base
3	13.97	24.2%	Mejora $\geq 15\%$ → continuar
4	11.68	16.4%	Mejora $\geq 15\%$ → continuar
5	10.30	11.8%	Mejora $< 15\%$ → PARAR → mejor K = 4

Figura 2. Evolución del WCD y mejora relativa al aumentar K en K-Means



La grafica deja ver bien la logica del criterio de parada: el WCD cae con fuerza al pasar de K=2 a K=3 (24.2%, por encima del umbral del 15%), y el siguiente paso a K=4 da un 16.4%, tambien por encima del umbral, asi que continuamos. Es al pasar a K=5 cuando la mejora cae al 11.8%, por debajo del 15%. Ahi `find_bestK` se detiene y se queda con K=4 como mejor valor.

- En `Kmeans.py` `find_bestK` usa un threshold del 15% (antes 20%), lo que lleva al mejor K=4.
- En `app.py` se usa `find_bestK(max_K=5)` con inicialización `kmeans++` y `garment_mask`.
- En `analysis.py` se compara K fijo (2, 3, 4, 5) con `find_bestK`.

4.3 Inicialización de centroides

K-Means es bastante sensible a los centroides iniciales: si arrancan mal, o el resultado final no es tan bueno, o el algoritmo tarda más en converger. Por eso la inicialización no es un detalle menor.

- `first`: usa los primeros píxeles como centroides. Rápido pero puede ser poco representativo.
- `random`: elige píxeles aleatorios. Puede funcionar bien pero no siempre da el mismo resultado.

- custom (kmeans++): separa los centroides iniciales maximizando distancias. Método por defecto en esta implementación.

Con una buena inicialización se consigue lo de siempre: menos iteraciones para converger y, en general, colores finales más representativos.

4.4 Características usadas en KNN

Al principio simplemente le pasábamos a KNN la imagen en escala de grises aplanada, un vector de 4 800 valores tal cual. Funcionaba, pero como mejora añadimos un descriptor de silueta (shape_features.py) que junta varias cosas:

- Máscara de primer plano redimensionada a 32×32 (separación fondo/prenda).
- Imagen en gris recortada al bounding-box, redimensionada a 32×32.
- Proyecciones de filas y columnas de la máscara.
- Descriptores escalares: aspect ratio, fill ratio, centro de masa, etc.

Este descriptor aguanta mucho mejor las variaciones de posición y de fondo, lo que se nota sobre todo en prendas pequeñas dentro de la imagen, como las Sandals o las Flip Flops, donde el método antiguo flaqueaba más.

5. Análisis de eficiencia

El análisis de eficiencia nos sirve para hacernos una idea realista de cuánto tarda el sistema y qué recursos consume, que es algo que conviene tener claro antes de pensar en escalar el proyecto.

5.1 Eficiencia de KNN

KNN tiene una particularidad muy cómoda: prácticamente no se entrena. Lo único que hace en esa fase es guardar las imágenes de entrenamiento, no calcula ningún modelo a partir de ellas.

- Entrenamiento: rápido, solo almacena los datos.
- Predicción: $O(N \cdot D)$ por muestra, donde N = imágenes train, D = dimensión feature.
- Memoria: proporcional al tamaño del dataset de entrenamiento.

5.2 Eficiencia de K-Means

K-Means opera directamente sobre los píxeles de cada imagen, que en nuestro caso son 4800 (60×80).

- Coste por imagen: $O(N \cdot K \cdot I)$, con N píxeles, K grupos e I iteraciones.
- Con K entre 2 y 6, el sistema es rápido para imágenes pequeñas.

5.3 Eficiencia de find_bestK

find_bestK lanza K-Means varias veces seguidas, una por cada valor de K que prueba. El coste extra existe, pero es asumible porque limitamos el máximo de K a un número pequeño (≤ 8).

5.4 Resumen de eficiencia

Parte	Ventaja	Limitación
KNN	No necesita entrenamiento complejo.	La predicción puede ser lenta con muchos datos.
K-Means	Funciona bien con imágenes pequeñas.	Depende de K y de las iteraciones.
find_bestK	Elige mejor el número de colores.	Ejecuta K-Means varias veces.

6. Conclusiones

El proyecto deja claro que no hace falta recurrir a algoritmos sofisticados para montar un sistema de etiquetado de imágenes de ropa que funcione razonablemente bien. Con KNN logramos reconocer la forma de la prenda y con K-Means resumimos sus colores principales, y eso ya es suficiente para cumplir el objetivo planteado.

- KNN funciona bien porque las imágenes están preparadas y tienen un formato parecido.
- K-Means es útil para resumir una imagen en pocos colores dominantes.
- La elección de K es importante en ambos algoritmos. Se ha elegido K=1 en KNN con shape features porque maximiza la accuracy (91.30%) sobre el conjunto de test, mejorando en +19.28pp respecto al método original con píxeles en gris (72.02% con K=5).

- Se ha usado accuracy como métrica porque el dataset está equilibrado y el coste de error es simétrico entre clases.
- El sistema es eficiente para datasets pequeños, pero podría volverse lento al escalar.

De cara al futuro, una línea de mejora interesante sería incorporar descriptores más potentes (por ejemplo HOG o descriptores de contorno) y combinarlos con métodos de búsqueda aproximada de vecinos como KD-Tree, que acelerarían bastante la predicción cuando el dataset crezca.

En definitiva, el sistema responde al objetivo que nos habíamos marcado: asignar etiquetas de forma y color a las imágenes de productos para hacer posibles búsquedas más inteligentes dentro de una tienda online.